

TAKE-HOME EXAM (VARIANT 1)

QUESTION 1 – MOVIE RATINGS

```
def parse_ratings():
    global data
    global singledata
    global ratinglist
    global movienames
    global grades
    global parseddata
    x = 0

    for data in data.split(","):
        singledata.append(data)

    for singledata in singledata:
        for minidat in singledata.split(":"):
            if len(minidat) > 3:
                movienames.append(minidat)
            else:
                minidat = int(minidat)
                grades.append(minidat)

    while x < len(movienames):
        temp1 = movienames[x]
        temp2 = grades[x]
        temp = tuple((temp1,temp2))
        parseddata.append(temp)
        x += 1

    return
```

To start the process of the Movie Ratings, we take the loaded data (*data*), and we split it into the individual movie:grade formats into the new list single data. Then, this data is cycled to divide the system into the movie names (*movienames*) and the grades (*grades*). Then both lists are joined into a tuple, who is then added into a list of movie names and grades named *parseddata*. Printing this *parseddata* gives us the list with the tuples.

```
movie = "Dune"
def average_rating(movie):
    global movienames
    global grades
    global avg

    y = 0
    num = 0
    preavg = 0

    while y < len(movienames):
        if movie == movienames[y]:
            num += 1
            preavg += grades[y]
            y += 1
        else:
            y += 1

    if num > 0:
        avg = preavg/num
        print(f"The average for {movie} list: {avg:.2}")
    else:
        print("Movie not on list. 0.0")

    return avg
```

For the average ratings, we take the *movienames* and *grades* data, and we use the input (default input: “Dune”) to check what movie are we looking for and add its grade to the value *preavg*, utilizing the listing cycle. During this process, we also create a variable *num* to count how many movies there are. Afterwards, we just divide the *preavg* and the *num* values to obtain the average. If the movie is not in the list, the *num* factor is zero, and thus the if function sends a “Movie not on list. 0.0” result.

```
def max_rating():
    vally = 0
    maxi = 0
    z = 0
    moviemax = []
    donemovies = []

    while z < len(movienames):
        if movienames[z] in donemovies:
            pass
        else:
            inner_average_rating(movienames[z])
            temp = avge
            if avge > maxi:
                maxi = avge
                moviemax = [movienames[z],avge]
            else:
                if avge == maxi:
                    moviemax[0].append(movienames[z])
                    vally += 1
                else:
                    pass
            donemovies.append(movienames[z])
            z += 1

    if vally != 1:
        print(f"The highest rated movie is {moviemax[0]} with {moviemax[1]}")
    else:
        print(f"The highest rated movies are {moviemax[0][0]} and {moviemax[0][1]} with {moviemax[1]}")
```

For the max rating function, we use a second average function (that doesn’t return a text) to calculate the average of the movie. Afterwards, we create a list of the movies and its averages by checking it against a list of movies already on the list. This value *avge* is compared to a value set as *maxi* and if its higher to the current value, it’s appended to *moviemax* along with the movie name. In case we have two movies with the same grades, it adds the value *vally* so the final if defines whether it’s one or two movies.

```
def ratings_count():
    global movienames
    n=0
    number = 0
    final = {}
    allmovs = []

    while n < len(movienames):
        if movienames[n] in allmovs:
            n += 1
            pass
        else:
            allmovs.append(movienames[n])
            temp = movienames.count(movienames[n])
            final.update({f'{movienames[n]}':temp})
            n += 1
            pass
    print(f"The movies received the following number of reviews: {final}.")
```

Finally, for the rating counts, we use the same check to see if the movie is already on the list. If it's not, the movie is counted with the .count function and its result and the movie name is added to a list as a tuple. Then the whole list is printed by itself.

Putting the following input:

```
parse_ratings()
print(parseddata)
#print(movienames)
#print(grades)
average_rating("Barbie")
max_rating()
ratings_count()
```

Gives us the following result on the terminal:

```
PS C:\Users\jdani\OneDrive\Documentos\SRH\Dev Class\Take Home Exam - 100006002 Juan Parra> & C:\Users\jdani\AppData\Roaming\uv\python\python.exe "c:/Users/jdani/OneDrive/Documentos/SRH/Dev Class/Take Home Exam - 100006002 Juan Parra/q1.py"
[('Dune', 8), ('Dune', 9), ('Barbie', 7), ('Dune', 10), ('Barbie', 9), ('Oppenheimer', 9), ('Barbie', 6)]
The average for Barbie is: 7.3
The highest rated movies are Dune and Oppenheimer with 9.0
The movies received the following number of reviews: {'Dune': 3, 'Barbie': 3, 'Oppenheimer': 1}.
```

QUESTION 2 - GRADEBOOK

```
subjects = grades.keys()

def subjects_of(student):
    global subjects
    global grades

    tempst = set()

    for subjects in grades:
        for st in grades[subjects]:
            if st == student:
                tempst.add(subjects)
            else:
                pass

    print(f"{student} takes {tempst}")
```

The first thing done in the program was to obtain a list consisting of the subjects to use in future functions. With these, we utilize them to loop the subjects, followed by the loop that checks every student in that subject. It uses the input to check whether to add the subject to the *tempst* set. Otherwise, it passes. Then this set is printed.

```
def takes_all():
    global subjects
    global grades
    global kids

    kids=set()
    notinclass = set()
    allclass = kids

    for subjects in grades:
        for st in grades[subjects]:
            kids.add(st)

    for subjects in grades:
        notinclass = kids
        students = grades[subjects].keys()
        notinclass = notinclass - students
        allclass = allclass - notinclass

    if len(allclass) == 0:
        print("No student is in all classes.")
    else:
        print (f"The student in all classes is: {allclass}")
```

For the *takes_all* function, we first create a set with all kids once called *kids* and one set called *notinclass* which is, at the beginning, equal to *kids*. Then, we make a for loop using the subjects, then we use the keys function to obtain a list of kids in that subject. That list is subtracted from the *notinclass* variables, thus giving us a list of students not in that subject. This list is subtracted from the *allinclass* variable, which is a list of students that go to all classes. If a student is in all classes, it never appears in *notinclass*. When all subjects are checked, this list is printed. However, if this list is empty, it prints "No student is in all classes."

```
def student_average(student):
    global grades
    global subjects

    x = 0
    num = 0
    preavg = 0

    for subjects in grades:
        for students in grades [subjects]:
            if students == student:
                temp = grades[subjects][students]
                preavg += temp
                num+=1
            else:
                pass

    if num > 0:
        avg = preavg/num
        print(f"The average for {student} is: {avg:.3}")
    else:
        print("Student not enrolled.")
```

For the student average, we use a similar method to the first exercise, but we add another level, where it checks the student within the subject. Dictionary checking is used to manage the nested dictionary.

```
def honor_roll(grade):
    global grades
    global subjects
    global kids

    num = 0
    reavg = 0
    tempi = 0
    kids=set()
    honorroll = []

    for subjects in grades:
        for st in grades[subjects]:
            kids.add(st)

    for student in kids:
        for subjects in grades:
            for students in grades[subjects]:
                if students == student:
                    tempi = grades[subjects][student]
                    reavg += tempi
                    num+=1
                else:
                    pass

        avg = reavg/num

        if avg <= grade:
            hr = [student,(round(avg,2))]
            honorroll.append(hr)
        else:
            pass

        avg=0
        reavg = 0
        num=0
        tempi = 0

    honorroll.sort()
    print(honorroll)
```

For the final exercise, the honor roll, we create first another list of all kids. With this list, we created nested loops that will calculate the average grade for everyone student. This calculated average is then added into *honorroll*, a list that includes all honor roll students. To make sure that only the students with the input grade enter, this append is inside an if that depends on the average obtained. Finally, the values are reset so future calculations are clean.

Outside the loop, values are sorted.

For the following input:

```
subjects_of("anna")
takes_all()
student_average("anna")
honor_roll(2)
```

We obtain the following result on the terminal:

```
PS C:\Users\jdani\OneDrive\Documents\SRH\Dev Class\Take Home Exam - 100006002 Juan Parra> & C:\Users\jdani\AppData\Roaming\uv\python\python.exe "c:/Users/jdani/OneDrive/Documents/SRH/Dev Class/Take Home Exam - 100006002 Juan Parra/q2.py"
anna takes {'art', 'math'}
No student is in all classes.
The average for anna is: 1.35
[['anna', 1.35], ['clara', 1.15], ['david', 1.85]]
```

QUESTION 3 – RECIPE SCALER

```
def scale_recipe(name,servings,*ingredients,unit="g",**options):
    listy = []
    num1 = 0
    x = 0
    y=0

    if servings < 1:
        print ("Minimum servings: 1.")
    else:
        print(f"\n\nFor {servings} unit(s) of {name}(s) you'll need:")
        while num1 < len(ingredients):
            temp1= ingredients[num1][0]
            temp2= (ingredients[num1][1]) * servings
            listy.append((temp1,temp2))
            num1 +=1

        while x < len(listy):
            print(f"-{listy[x][0]}: {listy[x][1]}{unit}")
            x+=1

        if len(options)!= 0:
            print("\nConsider the following:")
            for key in options:
                temp3 = options[key]
                print(f"{key}: {temp3}")

        else:
            pass
```

To generate the recipe, we start with an if that checks whether the servings are one of higher. It returns a minimum servings message. If the servings are higher than 1, then we start by calculating the ingredients by obtaining the values from the tuple and putting them into a list updates with the measurements related to the servings. Then this list is looped to print the readable listing. Finally, we take the options and print them as a list, with a colon in the middle.

For the following input:

```
scale_recipe("Licorice",0,("Licorice",1))
scale_recipe("Toast",1,("Bread",25))
scale_recipe("Pizza",3,("Dough",100),("Tomato Sauce",50),("Cheese",30),unit="oz")
scale_recipe("Marble Cake",10,("Flour",100),("Eggs",25),("Chocolate Powder",50),("Vanilla Extract",5),("Milk",200),
unit="g",bakefor="60 minutes",bakeat="175 C")
```

We obtain the following in the terminal:

```
PS C:\Users\jdani\OneDrive\Documents\SRH\Dev Class\Take Home Exam - 100006002 Juan Parra> & C:\Users\jdani\AppData\Roaming\uv\python\python.exe "c:/Users/jdani/OneDrive/Documents/SRH/Dev Class/Take Home Exam - 100006002 Juan Parra/q3.py"
Minimum servings: 1.

For 1 unit(s) of Toast(s) you'll need:
-Bread: 25g

For 3 unit(s) of Pizza(s) you'll need:
-Dough: 300oz
-Tomato Sauce: 150oz
-Cheese: 90oz

For 10 unit(s) of Marble Cake(s) you'll need:
-Flour: 1000g
-Eggs: 250g
-Chocolate Powder: 500g
-Vanilla Extract: 50g
-Milk: 2000g

Consider the following:
bakefor: 60 minutes
bakeat: 175 C
```

QUESTION 4 - LIBRARY

```
class Book:
    def __init__(self, title, author, year):
        self.title = title
        self.author = author
        self.year = int(year)

    def __str__(self):
        return f'{self.title} by {self.author} ({self.year})'

    def __repr__(self):
        return f'Title = {self.title}, Author = {self.author}, Year = {self.year}'

    def __eq__(self, other):
        self.title == other.title
        self.author == other.author

    def age(self):
        self.yearint = int(self.year)
        age = 2026 - self.yearint
        print(f'{self.title} is {age} years old.')
```

To create the class, we start the `__init__` with the three required characteristics: *title*, *author* and *year*. We represent it with a string ordering and typing it orderly. We make sure it can be compared with other iterations of itself, and, finally, we create the age function which subtracts it from the current year.

```
class Ebook:
    def __init__(self, title, author, year, size):
        Book.__init__(self, title, author, year)
        self.size = int(size)

    def __str__(self):
        return f'{self.title} by {self.author} ({self.year})(Size: {self.size}mb)'

    def __repr__(self):
        return f'Title = {self.title}, Author = {self.author}, Year = {self.year}, Size in Megabytes: {self.size}'

    def download_speed(self, mbitps):
        self.mbps = mbitps
        dispeed = self.size/self.mbps
        print(f'{self.title} will be downloaded in {dispeed:.3} seconds.')

    def age(self):
        self.yearint = int(self.year)
        age = 2026 - self.yearint
        print(f'{self.title} is {age} years old.')
```

For the ebook function., it's the same structure, but for the `__init__`, we include the size in megabytes. We calculate the download speed by dividing the size of the called ebook by an input of a speed given. Otherwise, we use the same syntax.

```
class Library:
    def __init__(self):
        self.collection = []

    def __repr__(self):
        return f'The Library has the following books: \n{self.collection}'

    def __len__(self):
        print(f'There are {len(self.collection)} book(s) in the library.')
        return len(self.collection)

    def add(self, book):
        loopy = 0
        for x in self.collection:
            if book.title == x.title:
                return print(f'{x.title} is already in the collection.')
            else:
                loopy += 1

        if loopy == len(self.collection):
            self.collection.append(book)
        else:
            pass
```

For the *Library* class, we start by initializing it with an empty *self.collection* and a representation of a listing with a proper heading. The length of the collection is calculated by a nested function, and it returns an ordered string. To add, we start by checking if the book is already in the collection by comparing the title with list of titles (via a loop).

For the following input:

```
MunicipalLibrary = Library()
book1 = Book("House of Leaves", "Mark Z. Danielewski", "2001")
book2 = Book("Beastars Vol. 1", "Paru Itagaki", "2017")
book3 = Book("Beastars Vol. 15", "Paru Itagaki", "2019")
book4 = Book("The Hound of the Baskervilles", "Arthur Conan Doyle", "1902")
book5 = Book("House of Leaves", "Mark Z. Danielewski", "2001")
ebook1 = EBook("Dorohedoro - Volume 1", "Q Hayashida", "2000", "250")
ebook2 = EBook("The Jojolands - Volume 1", "Hirohiko Araki", "2023", "300")
ebook3 = EBook("There is no antimemetics division", "QNTM", "2020", "450")
MunicipalLibrary.add(book1)
MunicipalLibrary.add(book2)
MunicipalLibrary.add(book3)
MunicipalLibrary.add(book4)
MunicipalLibrary.add(book5) #repeat book
MunicipalLibrary.add(ebook1)
MunicipalLibrary.add(ebook2)
MunicipalLibrary.add(ebook3)
print(book2)
print(ebook1)
book4.age()
ebook2.download_seconds(11)
MunicipalLibrary.find_by_author("Paru Itagaki")
MunicipalLibrary.oldest()
len(MunicipalLibrary)
```

We obtain in the terminal:

```
PS C:\Users\jdani\OneDrive\Documentos\SRH\Dev Class\Take Home Exam - 100006002 Juan Parra> & C:\Users\
.14.6-windows-x86_64-none\python.exe "c:/Users/jdani/OneDrive/Documentos/SRH/Dev Class/Take Home Exam
House of Leaves is already in the collection.
Beastars Vol. 1 by Paru Itagaki (2017)
Dorohedoro - Volume 1 by Q Hayashida (2000)(Size: 250mb)
The Hound of the Baskervilles is 124 years old.
The Jojolands - Volume 1 will be downloaded in 27.3 seconds.
The books written by Paru Itagaki are:
- Beastars Vol. 1
- Beastars Vol. 15
The oldest book is The Hound of the Baskervilles
There are 7 book(s) in the library.
```

QUESTION 5 – MOVIE REVIEW EXTRACTION PIPELINE

I couldn't move forward very much with the final question, as I had issues with the loading of the openai library, and the syntax for the AI implementation was something I didn't manage to master. In my desire to not use AI generation and to program all myself, I decided to accept the fact I will not be able to complete this question. I will continue studying the process and try to learn by myself. I apologize for the lack of results in this question.